

Piano di Implementazione: Classificazione Dinamica degli Archetipi VGC

Questo documento delinea il piano strutturale e matematico per l'aggiornamento dell'algoritmo di classificazione dei team VGC. L'obiettivo principale è attuare una transizione verso un'analisi basata **esclusivamente sull'Event Sourcing** per la determinazione dinamica degli archetipi. Il codice deve essere pronto e aperto all'implementazione di tecniche per il riconoscimento di nuovi archetipi.

1. Differenziazione tra Archetipo Statico e Dinamico

È fondamentale tracciare una netta differenziazione tra archetipo statico e archetipo dinamico:

- **Analisi Dinamica (Archetipo Dinamico):** Deriva dal comportamento nel match, ovvero come viene giocato il team sul campo da battaglia [cite: 2]. Analizza variabili comportamentali e i tag delle azioni di gioco (come salvate nelle tabelle Turn e TurnAction) [cite: 2]. Questa è la logica principale richiesta per l'implementazione attuale.
- **Analisi Statica (Archetipo Statico):** Deriva dall'analisi statica della composizione (chi c'è nel team) tramite gli indicatori strutturali estratti dalla tabella PokemonBuild (es. Abilità Chiave, Core Strutturali, Statistiche Base) [cite: 2]. **L'analisi statica dovrà essere implementata in futuro** per fornire una profilazione completa.

2. Filosofia dell'Algoritmo: Event Sourcing Puro (Fase Attuale)

Nel metagame competitivo, la presenza di una mossa o di un'abilità non garantisce il suo utilizzo. Un team con Torkoal (Drought) potrebbe non sfruttare mai il Sole se Torkoal viene eliminato prima di entrare, o se il clima viene costantemente sovrascritto. Per questo motivo, l'algoritmo misura l'**impatto effettivo** sul campo di battaglia.

- **Eliminazione dell'Analisi Statica (per ora):** L'algoritmo non interrogherà le mosse o le abilità note del team in questa iterazione.
- **Centralità dei Tag JSON:** L'identità di un team viene calcolata parsando le chiavi specifiche generate da Showdown (es. -weather, -boost, -setboost).
- **Aggregazione Percentuale:** Dato un raggruppamento di replay per lo stesso team, il sistema calcolerà la frequenza pesata di ciascun archetipo validato nei singoli match (es. *SunnyDay Team*: 60%).

3. Mappatura Algoritmica degli Archetipi (Dinamici)

Di seguito è spiegata la logica per individuare ciascun archetipo, dipendente al 100% dal tracking del comportamento in partita. Sia N il numero totale dei turni del match.

Archetipo	Origine dei Dati	Condizione Dinamica (Soglia)
Weather Team (Dinamico)	TurnAction.tags["-weather"] e attributi Attore (es. P1a/P2a) [cite: 4, 8]	Identificato se un clima specifico domina per almeno il 50% dei turni ($F_{Weather} \geq 0.5$) E SE il team in analisi è colui che ha settato il meteo (verificando l'attore nei tag o in TurnAction.actor_build_id) [cite: 4, 8]. Non assegnare mai questo archetipo a un team che si limita a subire il meteo avversario. Il nome prende il tag esatto (es. <i>SunnyDay Team</i>).
Setup Sweeper / Setter	TurnAction.tags["-boost"] o ["-setboost"]	Il numero di azioni di potenziamento (boost) diviso il numero totale dei turni N è ≥ 0.15 .
Hard Trick Room	Tabella Turn (flag <i>trick_room</i>)	La Distortozona è attiva per almeno il 40% dei turni del match ($F_{TrickRoom} \geq 0.4$).
Tailwind Offense	Tabella Turn (flag <i>p1_tailwind</i> / <i>p2_tailwind</i>)	Ventoincoda è attivo per almeno il 30% dei turni ($F_{Tailwind} \geq 0.3$) e il match si conclude in ≤ 7 turni totali (Hyper Offense).
Balance / Good Stuff	TurnAction.action_type == "switch"	Il tasso di switch in rapporto ai turni N è altissimo ($F_{sw} \geq 0.4$) e la partita si estende per lunghi scambi ($N \geq 8$ turni).

4. Priorità e Gerarchia di Assegnazione

Un singolo match potrebbe tecnicamente soddisfare più criteri contemporaneamente. Per evitare conflitti, l'algoritmo valuta le condizioni in ordine di dominanza decrescente:

1. **Trick Room:** È la condizione più polarizzante. Se verificata, il team è inevitabilmente costruito per sfruttarla.
2. **Setup Sweeper:** Se un team basa la sua strategia sull'accumulo sistematico di buff, questo definisce il suo nucleo tattico.

3. **Meteo (Dinamico):** Se non ci sono Setup o Trick Room, ma il sole/pioggia domina il campo ed è stato attivato attivamente dal team.
4. **Tailwind / Balance:** Indicatori di ritmo di gioco (Hyper Offense contro posizionamento lento).

5. Prompt per l'Agente AI (Antygraviti)

Copia il seguente prompt e passalo all'agente per implementare la logica nel codice Python:

Sei Antygraviti, un agente software esperto in Python e architetture di database relazionali (SQLAlchemy).

Il mio obiettivo è aggiornare la funzione che calcola l'archetipo di un team VGC analizzando un gruppo di replay.

ESTENSIBILITÀ E FUTURO:

- Struttura il codice prevedendo una netta differenziazione tra "Archetipo Dinamico" e "Archetipo Statico". L'archetipo statico deriva da un'analisi statica della composizione e dovrà essere implementato in futuro, quindi lascia lo spazio architeturale adeguato.
- Il codice deve essere pronto e aperto all'implementazione rapida di tecniche per il riconoscimento di nuovi archetipi futuri.

VINCOLO ASSOLUTO (PER LA FASE ATTUALE DINAMICA): L'archetipo attuale deve essere dedotto SOLO ED ESCLUSIVAMENTE da ciò che avviene in battaglia (analisi dinamica basata sui tag dei log e le condizioni dei turni). È severamente vietato in questo step leggere la composizione del team (mosse, abilità o specie presenti nella PokemonBuild) per dedurre l'archetipo.

Devi scrivere/aggiornare la funzione Python `analizza_archetipo_team(team_id, lista_match_ids, session)` seguendo queste regole:

1. METEO DINAMICO: Controlla la presenza della chiave "-weather" all'interno della colonna JSON `TurnAction.tags`.

IMPORTANTE: L'algoritmo deve verificare CHI ha settato il meteo leggendo gli attributi tag (es. stringhe contenenti "[of] p1a:", "[of] p2a:" all'interno dei dettagli dell'azione) o verificando l'attributo `TurnAction.actor_build_id`.

Assegna l'archetipo meteo AL TEAM IN ANALISI (es. "SunnyDay Team") SOLO SE il meteo dominante ($\geq 50\%$ dei turni) è stato effettivamente generato da uno dei Pokémon di quel team. Un team che non ha setter di meteo e che subisce semplicemente il meteo avversario NON deve ricevere l'archetipo meteo. Escludi sempre "none" e "ClearSkies".

2. SETUP SWEEPER: Controlla se le chiavi "-boost" o "-setboost" compaiono nei tag di `TurnAction`. Se il rapporto tra le azioni di boost auto-inflitte o da alleati e il totale dei turni (F_{boost}) è ≥ 0.15 , l'archetipo è "Setup Sweeper".

3. HARD TRICK ROOM: Conta i turni in cui `Turn.trick_room == True`. Se i turni sono $\geq 40\%$ (0.4), l'archetipo è "Hard Trick Room".

4. TAILWIND OFFENSE: Conta i turni in cui `Turn.p1_tailwind` o `Turn.p2_tailwind` sono True. Se $\geq 30\%$ (0.3) e il totale dei turni (N) è ≤ 7 , l'archetipo è "Tailwind Offense".

5. BALANCE: Conta le azioni dove `TurnAction.action\_type == "switch"`. Se gli switch divisi per i turni totali (N) sono $\geq 40\%$ (0.4) e $N \geq 8$, l'archetipo è "Balance".

Gerarchia di classificazione (if/elif): Hard Trick Room -> Setup Sweeper -> [Weather Dinamico] -> Tailwind Offense -> Balance.

L'output della funzione deve restituire una stringa riassuntiva che aggrega i risultati di tutti i match_ids analizzati, stampando le probabilità percentuali, nel formato esatto: "Team {team_id} : SunnyDayTeam:60% TrickRoom:30% SetupSweeper:10%".

Scrivi il codice ottimizzando le query per SQLAlchemy 2.0.